

PROJECT 02

Auto Reply Generator

A complete build log — what we planned, what broke, and how we fixed it

n8n · OpenAI · Gmail · Google Sheets

Difficulty Beginner — Intermediate	AI Node Basic LLM Chain	Time to Build 1 — 2 Hours
--	-----------------------------------	-------------------------------------

Legacy Restaurant · Ibadan, Nigeria · 2025

What We Set Out to Build

Every customer complaint was being ignored after it was logged. No one was writing back to upset customers — the feedback sheet just filled up with complaints that went unanswered.

The goal was to build a workflow that:

- Reads every complaint from the feedback sheet automatically
- Sends each complaint to an AI that writes a unique, personalised apology email
- Sends the email directly to the customer
- Tags each processed row so the workflow knows who has already been replied to

Why AI and not a template

A template sends the same email to everyone. A customer who complained about finding plastic in their food and a customer who complained about slow delivery get the exact same message. The AI reads what each customer actually said and writes a response specific to their complaint. Every email is different.

The Original Workflow Plan

This is what we planned before starting:

```
Manual Trigger
├─ Google Sheets (Get Rows)
│   └─ Filter (Rating ≤ 2)
│       └─ Loop Over Items
│           ├─ Basic LLM Chain (write reply)
│           │   └─ Send Email
│           │       └─ back to Loop input
│           └─ Done output → Edit Fields (Reply Sent = true)
```

The data source was the same Legacy Restaurant Feedback sheet from Project 01 — 50 rows total, 35 of which are complaints with Rating 1 or 2. Those 35 are the ones that need replies.

Step 1 — Manual Trigger

Add a Manual Trigger node. No configuration needed. This is the start button for the workflow.

Step 2 — Google Sheets (Get Rows)

Add a Google Sheets node. Operation: Get Rows. Connect your Google account, select the Legacy Restaurant Feedback sheet, leave the row limit blank to pull all 50 rows.

Step 3 — Filter Node

Add a Filter node. Condition: Rating (1-5) less than or equal to 2.

This drops all rows with Rating 3, 4, or 5 — only complaints proceed. After filtering, 35 items pass through.

Why Filter and not Switch

We only need a yes or no decision here — is this a complaint or not? Filter is the right tool for binary decisions. Switch is for routing to multiple different outputs. Since everyone who passes the filter gets the same treatment (an apology email), Filter is simpler and cleaner.

Step 4 — Loop Over Items

Add a Loop Over Items node.

This node is essential here. Without it, the Basic LLM Chain would receive all 35 complaints at once and potentially write one combined response instead of 35 individual ones. The Loop ensures each complaint is processed separately — one customer, one email, one at a time.

Items output (top)	Sends one item at a time to the next node
Done output (bottom)	Fires once after all items have been processed
Loop input (left)	Where processed items return to trigger the next iteration

Step 5 — Basic LLM Chain

Add a Basic LLM Chain node. Connect an OpenAI Chat Model (gpt-4o-mini) inside it. Toggle OFF Require Specific Output Format.

First attempt — plain text prompt

We started with a simple prompt asking the AI to write a personalised apology email. The output was good — the AI referenced the specific complaint and wrote a unique response for each customer.

But when the email arrived in the inbox it looked like this:

Error

Plain unformatted text. No colours, no structure, no visual hierarchy. Just a wall of text that looked unprofessional.

How we fixed it

Updated the prompt to instruct the AI to return clean HTML instead of plain text. Also changed the Send Email node body format from Text to HTML.

The lesson

When you want styled emails from an LLM, tell it explicitly to return HTML. Specify the exact tags, colours, and structure you want. The AI will follow precise HTML instructions reliably.

Second attempt — HTML but no visual impact

The email arrived with basic HTML formatting but still looked plain — no colours, no header, no visual identity. It did not look like it came from a restaurant brand.

How we fixed it

Added explicit HTML structure to the prompt — a full-width red header div with Legacy Restaurant branding, body with proper paragraph spacing, and a lighter coloured signature. The AI was given the exact HTML structure to follow.

The final prompt

After both fixes, here is the final prompt that produces branded, well-formatted apology emails:

```
You are a manager at Legacy Restaurant in Ibadan, Nigeria.

Write a warm, personalised apology email and return it as clean HTML only – no
markdown,
no backticks, no explanation. Just the HTML. Do not include <html>, <head>, or
<body>
tags – start directly from the content.

Use this exact HTML structure:

1. Header: a full-width div with background color #c0392b, padding 30px, centered
   white text saying "Legacy Restaurant" in bold 24px Arial

2. Body: a div with padding 30px, font Arial, color #333333
   - Greeting in bold
   - Body paragraphs in <p> tags with line-height: 1.8
   - Signature in color #777777
```

```
Write to {{ $json['Customer Name'] }} who said:  
"{{ $json['Comment'] }}"
```

```
Their rating was {{ $json['Rating (1-5)'] }} out of 5.
```

Rules:

- Do not include a subject line
- Start with the header div, then the body div
- Greeting: `<p><strong>Hi {{ $json['Customer Name'] }},</strong></p>`
- Acknowledge their specific complaint in your own words
- Apologise sincerely
- Tell them what action will be taken
- End with:
`<p style="color: #777777;">Warm regards,<br/>Restaurant Manager,<br/>Legacy Restaurant,<br/>Ibadan, Oyo State, Nigeria.</p>`

Keep it under 120 words. Be specific to what they complained about. No generic phrases.

Step 6 — Send Email

Add a Send Email node. Connect it after the Basic LLM Chain.

To	{{ \$('Filter').item.json.Email }}
Subject	We're Sorry About Your Experience, {{ \$('Filter').item.json['Customer Name'] }}
Body	{{ \$json['text'] }}
Body Format	HTML — not Text
Credential	Your Gmail credential

Important

The body format must be set to HTML, not Text. If it is set to Text, the HTML tags will appear as raw text in the email instead of rendering as formatted content.

Connecting the Loop

This is where most confusion happened during the build. The Loop node needs two connections going out and one coming back.

```
Loop Over Items  
├→ Items output (top dot) → Basic LLM Chain → Send Email → back to Loop input  
└→ Done output (bottom dot) → Edit Fields
```

How to make the connections:

1. Drag from the top output dot of Loop → connect to Basic LLM Chain
2. Basic LLM Chain already connects to Send Email
3. Drag from the Send Email output → drop onto the Loop Over Items node — it snaps to the loop back input on the left side
4. Drag from the bottom output dot of Loop → connect to Edit Fields

Why you connect Send Email back to the Loop

The Loop works like a conveyor belt. It sends one item, waits for it to come back, then sends the next. If Send Email does not connect back to the Loop input, the Loop never knows the item is done and stops after the first customer. Connecting it back is what makes all 35 emails send.

Step 7 — Edit Fields

Add an Edit Fields node. Connect it to the Done output (bottom dot) of the Loop Over Items node.

Configuration:

5. Click Add Field
6. Name: Reply Sent
7. Value: true

This fires once after all 35 emails have been sent. It tags every processed item with Reply Sent = true so you can later add a filter at the start of the workflow to skip customers who have already received a reply.

Side Problem — Gmail OAuth Invalid Client Secret

Before building the workflow we needed to set up Gmail credentials in n8n. The OAuth connection kept failing.

Error

```
Client authentication failed. {"error":"invalid_client","error_description":"The provided client secret is invalid."}
```

The Client Secret had been copied correctly from Google Cloud Console but was still being rejected. This happens when the secret has become invalid or was never saved properly after the OAuth client was created.

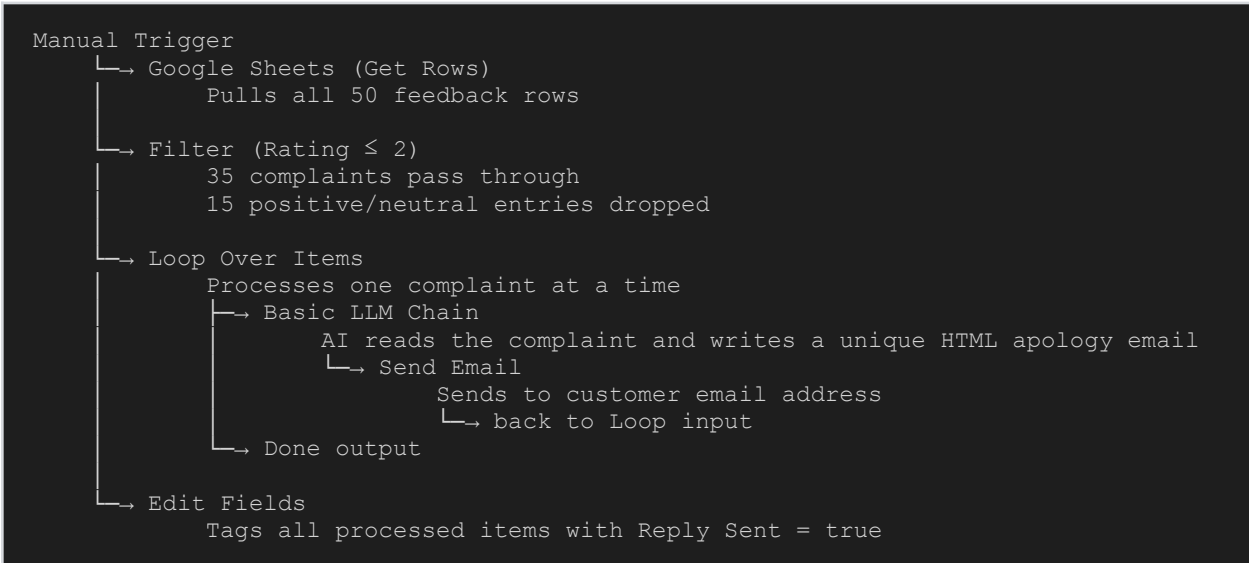
How we fixed it

Regenerated a new Client Secret in Google Cloud Console — APIs & Services → Credentials → click the OAuth client → Reset Secret. Copied the new secret immediately using the copy button and pasted it into n8n. The connection worked on the first attempt after that.

The lesson

If your Client Secret is rejected even though you copied it correctly, reset it in Google Cloud Console and get a fresh one. Do not try to reuse an old secret — once it is invalid, no amount of re-copying will fix it.

Final Workflow — What We Built



Result

35 unique personalised apology emails sent automatically. Every email references the customer's specific complaint. All emails arrive with Legacy Restaurant branding — red header, clean body, professional signature. Zero manual writing required.

Key Concepts From This Build

Concept	What it means
---------	---------------

Loop Over Items	Processes items one at a time. Has three connection points — items output (top), done output (bottom), and loop input (left). Send Email must connect back to the loop input to process all items.
Loop back connection	After the last node in the loop, connect back to the Loop input. This tells the Loop the item is done and to send the next one. Without this connection only the first item is processed.
Done output	The bottom output of the Loop node. Fires once after all items have been processed. Connect your final clean-up nodes here — Edit Fields, Update Row, notifications, etc.
HTML emails from LLM	Tell the AI to return HTML explicitly in the prompt. Specify exact tags, colours, and structure. Change the Send Email body format to HTML. Without both changes the email will not render correctly.
AI personalisation	Dynamic expressions like <code>{{ \$json['Customer Name'] }}</code> and <code>{{ \$json['Comment'] }}</code> inside the LLM prompt inject real customer data into the AI instruction. Every customer gets a unique email because the AI reads their actual complaint.
Reset Client Secret	If Gmail OAuth fails with <code>invalid_client</code> , regenerate the secret in Google Cloud Console rather than trying to reuse the old one. A reset secret solves the problem immediately.
Edit Fields	Adds or modifies fields on items inside n8n. Does not write to Google Sheets — use Update Row for that. Used here to tag items as processed after the loop completes.

Portfolio Assets Checklist

Before moving to Project 03, capture these:

- Screenshot of completed n8n canvas showing all nodes and loop connections
- Screenshot of one email in your inbox showing the red Legacy Restaurant header and personalised content
- Screenshot showing two different emails side by side to demonstrate AI personalisation
- Export workflow JSON from n8n — three dots menu → Download
- Upload to GitHub under `project-02-auto-reply/`